Python OOP – Practice Set

This set is based on the topics we've covered so far:

- **Introduction to OOP**
- **Classes and Objects in Python**
- **Constructors in Python**
- **Instance and Class Attributes**
- **Inheritance and Polymorphism**
- **Method Overriding and Operator Overloading**

These are simple starter questions to get you comfortable with object-oriented programming.

---

1. Create a Simple Class and Object

Create a class `Car` with a method `drive()` that prints `"Car is moving"`.
Create an object of `Car` and call `drive()`.

---

2. Constructor and Attributes

Create a class `Person` with a constructor (`__init__`) that accepts `name` and `age` as arguments and stores them as instance attributes.
Create an object and print the person's name and age.

---

3. Simple Inheritance

Create a base class `Animal` with a method `sound()` that prints `"Some sound"`.
Create a derived class `Dog` that overrides `sound()` to print `"Bark!"`.
Create an object of `Dog` and call `sound()`.

Python Advanced Concepts – Practice Set

This set is based on the topics we've covered so far:

- **Decorators in Python**
- **Getters and Setters**
- **Static & Class Methods**
- **Magic/Dunder Methods**
- **Exception Handling and Custom Errors**
- **map(), filter(), and reduce()**
- **Walrus Operator**
- **args and kwargs**

These exercises are designed to take your Python skills to the next level by practicing object-oriented and functional programming features.

---

1. Decorators in Python

1. Write a decorator `logger` that prints `"Function is being called"` before the function runs. Use it to decorate a function `say_hello()` that prints `"Hello!"`.
2. Write a decorator `timer` that calculates how long a function takes to execute. Test it with a function that sums numbers from 1 to 1,000,000.

---

2. Getters and Setters

1. Create a class `Employee` with a private attribute `_salary`.
    1. Use `@property` to define a getter for `salary`.
    2. Use `@salary.setter` to prevent setting negative values (print a warning instead).
    3. Create an object and test by setting positive and negative salaries.

---

3. Static & Class Methods

1. Create a class `MathUtils` with:
    1. A `@staticmethod` called `add(a, b)` that returns `a + b`.
    2. A `@classmethod` called `description(cls)` that prints `"This is a utility class for math operations."`
2. Call both methods without creating an object.

---

4. Magic/Dunder Methods

1. Create a class `Book` with attributes `title` and `author`.
    1. Implement `__str__()` so that printing the object displays `"Title by Author"`.
    2. Implement `__len__()` so that `len(book)` returns the length of the title.
2. Create two `Book` objects and test these methods.

---

5. Exception Handling and Custom Errors

1. Write a program that asks the user to enter a number and handles:
    1. `ValueError` if the input is not a number
    2. `ZeroDivisionError` if you try to divide by zero
2. Create a custom exception `NegativeNumberError` and raise it when the user enters a negative number.

---

6. map(), filter(), and reduce()

1. Use `map()` to convert `[1, 2, 3, 4, 5]` into their cubes.
2. Use `filter()` to get only even numbers from `[10, 11, 12, 13, 14]`.
3. Use `reduce()` from `functools` to find the product of all elements in `[1, 2, 3, 4]`.

---

7. Walrus Operator

1. Use the walrus operator to read input until the user enters `"quit"`. Print each input as it is entered.
2. Use the walrus operator in a list comprehension to store lengths of words from `["python", "rocks", "ai"]` in a list while filtering out words shorter than 4 characters.

---

8. *args and **kwargs

1. Write a function `sum_all(*args)` that accepts any number of integers and returns their sum.
2. Write a function `print_details(**kwargs)` that prints key-value pairs passed as arguments, for example:

```
print_details(name="Alice", age=25, city="Delhi")
# Output:
# name: Alice
# age: 25
# city: Delhi
```

---

9. Bonus Challenges

1. Combine a decorator with `*args` and `**kwargs` support so it can wrap any function regardless of its parameters.
2. Implement `__add__` in a `Vector` class so that adding two `Vector` objects returns a new `Vector` with summed components.
3. Create a small program where invalid user input raises a custom exception, logs the error, and continues execution instead of crashing.

Python Practice Set 1 (Beginners)

Welcome to your first Python practice set!
This set is based on the topics we've covered so far: installation, syntax, variables, typecasting, user input, comments, and operators.
Try to solve each problem on your own before looking at the solution.

Q1: Your First Program

Write a program that prints:

```
Hello, World! Welcome to Python.
```

Q2: Print a Poem

Write a program that prints the following poem using a single `print()` statement:

```
Twinkle, twinkle, little star,
How I wonder what you are!
```

(Hint: Use `\n` for a new line.)

Q3: Variables & Data Types

Create variables to store: - Your name (string)
- Your age (integer)
- Your height in meters (float)
- A boolean value representing whether you are a student

Print all of them in one line.

Q4: Typecasting Practice

You are given a string:

```
num = "45"
```

Convert it into an integer and add `10` to it. Print the result.

Q5: Taking User Input

Write a program that:

1. Asks the user for their favorite food.
2. Prints:

```
Wow! I also like <food>.
```

Q6: Simple Calculator

Write a program that:

1. Takes two numbers as input from the user.

2. Prints their **sum, difference, product, and quotient**.

Q7: Escape Sequences

Print the following output using escape sequences:

```
Harry said, "Python is awesome!"
This is on a new line.
This is a tab ->    <- here
```

Q8: Operator Challenge

Write a program that:

1. Takes an integer as input from the user.
2. Prints the **square** and **cube** of that number.

Q9: Quick Quiz (True/False)

Mark **True** or **False**:

1. Python code must always end with a semicolon `;`.
2. The `#` symbol is used for comments in Python.
3. `"123"` and `123` are the same in Python.
4. The `*` operator is used for multiplication.
5. `\n` creates a new line.
6. Variables in Python can start with numbers.
7. `int("10") + 5` gives `15`.